



<State>

[UNIFEI | Universidade Federal de Itajubá](#)

Instituto de Ciências Tecnológicas

ECO – <Mobile>

eduardo.felipe@unifei.edu.br

Gerenciamento de variáveis

Hooks

State

Props

Hooks

- São funcionalidades que podem ser adicionadas ao componente para realizar eventos específicos.

Exemplos:

- `useState`
 - Permite gerenciar estado em informações do componente. De outra forma, o modelo stateless resetaria as informações a cada atualização (refresh) do componente.
- `useEffect`
 - Permite disparar funções quando o componente é montado ou quando determinada dependência é alterada (ciclo de vida).



useState

- Esse Hook retorna **dois elementos**:
 - A inicialização de uma *variável*
 - Uma *função* do tipo “set” para manipular a variável

- Exemplo:

```
const [count, setCount] = useState(0);
```



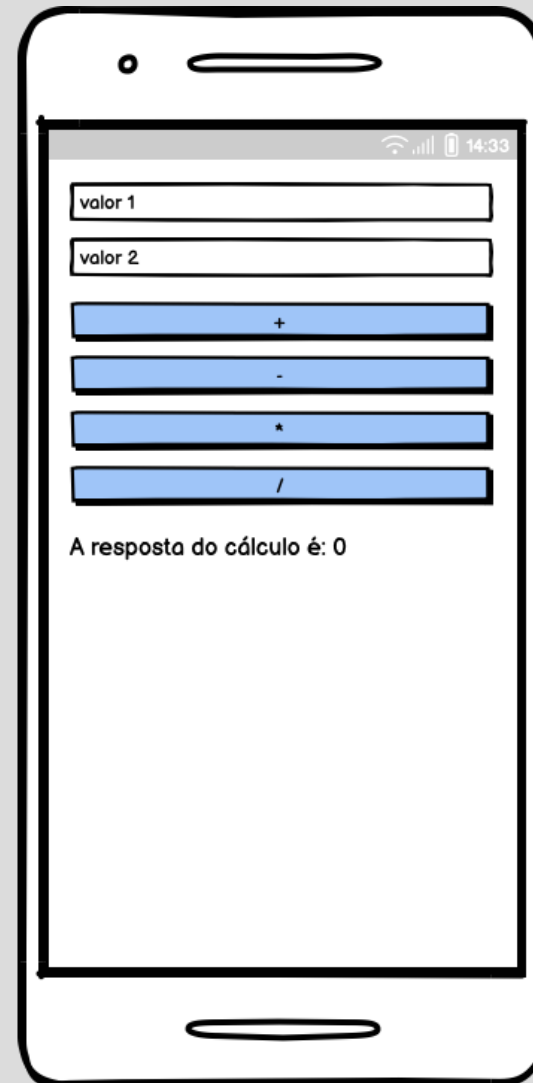
Exemplo – Contador.tsx

```
1 // React Native Counter Example using Hooks!
2
3 import React, {useState} from 'react';
4 import {View, Text, Button, StyleSheet} from 'react-native';
5
6 const Contador = () => {
7   const [count, setCount] = useState(0);
8
9   return (
10     <View style={styles.container}>
11       <Text>Você clicou {count} vezes</Text>
12       <Button onPress={() => setCount(count + 1)} title="Clique aqui" color={"black"} />
13     </View>
14   );
15 };
16
17 // React Native Styles
18 const styles = StyleSheet.create({
19   container: {
20     flex: 0,
21     alignItems: "center",
22     height: 50,
23     backgroundColor: "gray",
24   },
25 });
26
27 export default Contador;
```



Exercício

Calculadora



Exemplo – Index.tsx (ou .jsx)

```
import { Alert, Button, StyleSheet, Text, TextInput, View } from 'react-native'
```

```
import React, { useState } from 'react'
```

```
import { SafeAreaView } from 'react-native-safe-area-context'
```

```
export default function index() {
```

```
  const [valor1, setValor1] = useState('');
```

```
  const [valor2, setValor2] = useState('');
```

```
  const [resp, setResp] = useState(0);
```



```
export default function index() {  
  const [valor1, setValor1] = useState("");  
  const [valor2, setValor2] = useState("");  
  const [resp, setResp] = useState(0);  
  
  function calc(op: string) {  
    if ((valor1) && (valor2)) {  
      switch (op) {  
        case '+': setResp(parseFloat(valor1) + parseFloat(valor2));  
          break;  
        case '-': setResp(parseFloat(valor1) - parseFloat(valor2));  
          break;  
        case '*': setResp(parseFloat(valor1) * parseFloat(valor2));  
          break;  
        case '/': setResp(parseFloat(valor1) / parseFloat(valor2));  
          break;  
      }  
    } else {  
      Alert.alert("Favor inserir dois valores numéricos para o cálculo.")  
    }  
  }  
  
  return (  

```



Exemplo – Index.tsx

```
return (  
  <SafeAreaView>  
    <TextInput style={styles.caixa} keyboardType="numeric" placeholder="valor 1" value={valor1}  
onChangeText={(data) => setValor1(data)} />  
    <TextInput style={styles.caixa} keyboardType="numeric" placeholder="valor 2" value={valor2}  
onChangeText={(data) => setValor2(data)} />  
    <View style={styles.containerBotoes}>  
      <Button title='+' onPress={() => calc('+')} />  
      <Button title='-' onPress={() => calc('-')} />  
      <Button title='*' onPress={() => calc('*')} />  
      <Button title='/' onPress={() => calc('/')} />  
    </View>  
    <Text>A resposta do cálculo é: {resp}</Text>  
  </SafeAreaView>  
)  
}
```



Exemplo – Index.jsx

```
</SafeAreaView>  
)  
}
```

```
let styles = StyleSheet.create({  
  containerBotoes: {  
    margin: 10,  
  },  
  caixa: {  
    borderColor: "black",  
    borderStyle: "solid",  
    borderWidth: 1,  
    padding: 10,  
    margin: 5,  
  },  
  botao: {  
    margin: 3,  
    width: 50,  
  }  
})
```



```
index.tsx M X
app > (tabs) > index.tsx > index
1 import { Alert, Button, StyleSheet, Text, TextInput, View }
  from 'react-native'
2 import React, { useState } from 'react'
3 import { SafeAreaView } from 'react-native-safe-area-context'
4
5 export default function index() {
6   const [valor1, setValor1] = useState('');
7   const [valor2, setValor2] = useState('');
8   const [resp, setResp] = useState(0);
9
10  function calc(op: string) {
11    if ((valor1) && (valor2)) {
12      switch (op) {
13        case '+': setResp(parseFloat(valor1) + parseFloat(valor2));
14                  break;
15        case '-': setResp(parseFloat(valor1) - parseFloat(valor2));
16                  break;
17        case '*': setResp(parseFloat(valor1) * parseFloat(valor2));
18                  break;
19        case '/': setResp(parseFloat(valor1) / parseFloat(valor2));
20                  break;
21      }
22    } else {
23      Alert.alert("Favor inserir dois valores numéricos para o cálculo.")
24    }
25  }
26
27  return (
28    <SafeAreaView>
29      <TextInput style={styles.caixa} keyboardType="numeric"
30        placeholder="valor 1" value={valor1} onChangeText=
31        {(data) => setValor1(data)} />
32      <TextInput style={styles.caixa} keyboardType="numeric"
33        placeholder="valor 2" value={valor2} onChangeText=
34        {(data) => setValor2(data)} />
35      <View style={styles.containerBotoes}>
36        <Button title='+' onPress={() => calc('+')} />
37        <Button title='-' onPress={() => calc('-')} />
38        <Button title='*' onPress={() => calc('*')} />
39        <Button title='/' onPress={() => calc('/')} />
40      </View>
41      <Text>A resposta do cálculo é: {resp}</Text>
42    </SafeAreaView>
43  )
44 }
```

```
index.tsx M X
app > (tabs) > index.tsx > index > calc
5 export default function index() {
10  function calc(op: string) {
18    (valor2));
19    break;
20    case '/': setResp(parseFloat(valor1) / parseFloat(valor2));
21    break;
22  } else {
23    Alert.alert("Favor inserir dois valores numéricos para o cálculo.")
24  }
25 }
26
27 return (
28   <SafeAreaView>
29     <TextInput style={styles.caixa} keyboardType="numeric"
30       placeholder="valor 1" value={valor1} onChangeText=
31       {(data) => setValor1(data)} />
32     <TextInput style={styles.caixa} keyboardType="numeric"
33       placeholder="valor 2" value={valor2} onChangeText=
34       {(data) => setValor2(data)} />
35     <View style={styles.containerBotoes}>
36       <Button title='+' onPress={() => calc('+')} />
37       <Button title='-' onPress={() => calc('-')} />
38       <Button title='*' onPress={() => calc('*')} />
39       <Button title='/' onPress={() => calc('/')} />
40     </View>
41     <Text>A resposta do cálculo é: {resp}</Text>
42   </SafeAreaView>
43 )
44 }
45
46 let styles = StyleSheet.create({
47   containerBotoes: {
48     margin: 10,
49   },
50   caixa: {
51     borderColor: "black",
52     borderStyle: "solid",
53     borderWidth: 1,
54     padding: 10,
55     margin: 5,
56   },
57   botoes: {
58     margin: 3,
59     width: 50,
60   }
61 })
62 }
```



State x Props

- Props são variáveis que permitem passar dados de um componente (pai) para outro (filho).
- State também está relacionado a uma variável, não para passar parâmetros, mas para permitir o controle de dados no componente.





eduardo.felipe@unifei.edu.br